

7-Variable-magnitude

August 19, 2025

1 Variable magnitude

[Feature Engineering for Machine Learning Course](#)

1.1 Does the magnitude matter?

In linear regression models, the scale of variables used to estimate the output matters. Linear models predict y as follows: $\mathbf{y} = \mathbf{w} \mathbf{x} + \mathbf{b}$, where the regression coefficient w represents the expected change in y for a unit change in x (the predictor). Thus, the magnitude of w is partly determined by the magnitude of the units being used for x . If x is a distance variable, just changing the scale from kilometers to miles will cause a change in the magnitude of the coefficient.

In addition, when we estimate the outcome y using multiple predictors $x_1, x_2, \dots, x_n$, variables with greater numeric ranges dominate those with smaller numeric ranges.

Gradient descent converges faster when all the predictors (x_1 to x_n) are in a similar scale. Thus, having features in a similar scale is useful for neural networks.

In support vector machines, feature scaling can decrease the time it takes to find the support vectors.

Finally, methods using Euclidean distances or distances in general are also affected by the magnitude of the features, as Euclidean distance is sensitive to variations in the magnitude or scale of the predictors. Therefore, feature scaling is required for methods that utilise distance calculations like k-nearest neighbours (KNN) and k-means clustering.

In summary:

1.1.1 Magnitude matters because:

- The regression coefficient is directly influenced by the scale of the variable.
- Variables with a larger magnitude dominate those with a smaller magnitude.
- Gradient descent converges faster when features are on similar scales.
- Feature scaling helps decrease the time it takes to find support vectors for SVMs.
- Euclidean distances are sensitive to feature magnitude.

1.1.2 The machine learning models affected by the feature magnitude are:

- Linear and Logistic Regression.

- Neural Networks.
- Support Vector Machines (SVMs).
- KNN.
- K-means clustering.
- Linear Discriminant Analysis (LDA).
- Principal Component Analysis (PCA).

1.1.3 Machine learning models insensitive to feature magnitude are the ones based on trees:

- Classification and Regression Trees.
- Random Forests (RF).
- Gradient Boosted Trees.

1.2 In this Demo

We will study the effect of feature magnitude on the performance of different machine learning models.

We will use the Titanic dataset.

- To download the dataset please refer to the **Datasets** lecture in **Section 2** of the course.

```
[1]: import pandas as pd

# import several machine learning algorithms
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import AdaBoostClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.svm import SVC
from sklearn.neural_network import MLPClassifier
from sklearn.neighbors import KNeighborsClassifier

# to scale the features
from sklearn.preprocessing import MinMaxScaler

# to evaluate performance and separate into
# train and test set
from sklearn.metrics import roc_auc_score
from sklearn.model_selection import train_test_split
```

1.2.1 Load data

```
[2]: # Load numerical variables of the Titanic Dataset
```

```
data = pd.read_csv('../Datasets/titanic.csv',  
                    usecols=['pclass', 'age', 'fare', 'survived'])  
data.head()
```

```
[2]:
```

	pclass	survived	age	fare
0	1	1	29.0000	211.3375
1	1	1	0.9167	151.5500
2	1	0	2.0000	151.5500
3	1	0	30.0000	151.5500
4	1	0	25.0000	151.5500

```
[3]: # Let's have a look at the variables' values and  
# compare the feature magnitudes.
```

```
data.describe()
```

```
[3]:
```

	pclass	survived	age	fare
count	1309.000000	1309.000000	1046.000000	1308.000000
mean	2.294882	0.381971	29.881135	33.295479
std	0.837836	0.486055	14.413500	51.758668
min	1.000000	0.000000	0.166700	0.000000
25%	2.000000	0.000000	21.000000	7.895800
50%	3.000000	0.000000	28.000000	14.454200
75%	3.000000	1.000000	39.000000	31.275000
max	3.000000	1.000000	80.000000	512.329200

The variable Fare varies between 0 and 512. The variable Age varies between 0 and 80. The variable Class varies between 0 and 3. So the variables have different magnitude.

```
[4]: # Let's calculate the range.
```

```
for col in ['pclass', 'age', 'fare']:  
    print(col, 'range: ', data[col].max() - data[col].min())
```

```
pclass range: 2  
age range: 79.8333  
fare range: 512.3292
```

The range of values of each variable is different.

```
[5]: # Let's separate the data into training and testing sets.
```

```
# The titanic dataset contains missing information.  
# For this demo, I will fill in those values with 0s.
```

```

X_train, X_test, y_train, y_test = train_test_split(
    data[['pclass', 'age', 'fare']].fillna(0),
    data.survived,
    test_size=0.3,
    random_state=0)

X_train.shape, X_test.shape

```

```
[5]: ((916, 3), (393, 3))
```

1.3 Feature Scaling

For this demonstration, I will scale the features between 0 and 1, using the [MinMaxScaler](#) from Scikit-learn.

The transformation is given by:

$$X_rescaled = X - X.min() / (X.max() - X.min())$$

And to transform the re-scaled features back to their original magnitude:

$$X = X_rescaled * (max - min) + min$$

There is a section dedicated to feature scaling later in the course, where I will explain this and other scaling techniques in more detail.

For now, let's carry on with the demonstration.

```

[6]: # Scale the features between 0 and 1.

# The scaler.
scaler = MinMaxScaler()

# Fit the scaler.
scaler.fit(X_train)

# Re-scale the datasets.
X_train_scaled = scaler.transform(X_train)
X_test_scaled = scaler.transform(X_test)

```

```

[7]: # Let's have a look at the scaled training set.

print('Mean: ', X_train_scaled.mean(axis=0))
print('Standard Deviation: ', X_train_scaled.std(axis=0))
print('Minimum value: ', X_train_scaled.min(axis=0))
print('Maximum value: ', X_train_scaled.max(axis=0))

```

```

Mean: [0.64628821 0.33048359 0.06349833]
Standard Deviation: [0.42105785 0.23332045 0.09250036]
Minimum value: [0. 0. 0.]
Maximum value: [1. 1. 1.]

```

The maximum values of all features is 1, and the minimum value is 0, as expected.

1.3.1 Logistic Regression

Let's evaluate the effect of feature scaling in a logistic regression.

```
[8]: # Model trained with unscaled variables.

# The model.
logit = LogisticRegression(
    random_state=44,
    C=1000, # c big to avoid regularization
    solver='lbfgs')

# Train the model.
logit.fit(X_train, y_train)

# Evaluate performance.
print('Train set')
pred = logit.predict_proba(X_train)
print('Logistic Regression roc-auc: {}'.format(
    roc_auc_score(y_train, pred[:, 1])))
print('Test set')
pred = logit.predict_proba(X_test)
print('Logistic Regression roc-auc: {}'.format(
    roc_auc_score(y_test, pred[:, 1])))
```

```
Train set
Logistic Regression roc-auc: 0.6793181006244372
Test set
Logistic Regression roc-auc: 0.7175488081411426
```

```
[9]: # Let's look at the coefficients.
logit.coef_
```

```
[9]: array([[ -0.71428242, -0.00923013,  0.00425235]])
```

```
[10]: # Model trained with scaled variables.

# The model.
logit = LogisticRegression(
    random_state=44,
    C=1000, # c big to avoid regularization
    solver='lbfgs')

# Train the model using the re-scaled data.
logit.fit(X_train_scaled, y_train)

# Evaluate performance.
```

```

print('Train set')
pred = logit.predict_proba(X_train_scaled)
print('Logistic Regression roc-auc: {}'.format(
    roc_auc_score(y_train, pred[:, 1])))
print('Test set')
pred = logit.predict_proba(X_test_scaled)
print('Logistic Regression roc-auc: {}'.format(
    roc_auc_score(y_test, pred[:, 1])))

```

```

Train set
Logistic Regression roc-auc: 0.6793281640744896
Test set
Logistic Regression roc-auc: 0.7175488081411426

```

```
[11]: # Let's look at the coefficients.
```

```
logit.coef_
```

```
[11]: array([[ -1.42875872,  -0.68293349,   2.17646757]])
```

The performance of logistic regression did not change when using the datasets with the features scaled (compare ROC-AUC values for train and test set for models with and without feature scaling).

However, when looking at the coefficients, we do see a big difference in the values. This is because the magnitude of the variable affects the coefficients.

After scaling, all 3 variables have similar effect (coefficient) on survival, whereas before scaling, we would be inclined to think that Class was driving the survival outcome.

1.3.2 Support Vector Machines

```
[12]: # Model trained unscaled variables.
```

```

# The model.
SVM_model = SVC(random_state=44, probability=True, gamma='auto')

# Train the model.
SVM_model.fit(X_train, y_train)

# Evaluate performance.
print('Train set')
pred = SVM_model.predict_proba(X_train)
print('SVM roc-auc: {}'.format(roc_auc_score(y_train, pred[:, 1])))
print('Test set')
pred = SVM_model.predict_proba(X_test)
print('SVM roc-auc: {}'.format(roc_auc_score(y_test, pred[:, 1])))

```

```

Train set
SVM roc-auc: 0.882393490960506

```

Test set
SVM roc-auc: 0.6617581992146452

```
[13]: # Model trained with scaled variables.

# The model.
SVM_model = SVC(random_state=44, probability=True, gamma='auto')

# Train the model.
SVM_model.fit(X_train_scaled, y_train)

# Evaluate performance.
print('Train set')
pred = SVM_model.predict_proba(X_train_scaled)
print('SVM roc-auc: {}'.format(roc_auc_score(y_train, pred[:, 1])))
print('Test set')
pred = SVM_model.predict_proba(X_test_scaled)
print('SVM roc-auc: {}'.format(roc_auc_score(y_test, pred[:, 1])))
```

Train set
SVM roc-auc: 0.6780802962679695
Test set
SVM roc-auc: 0.6841435761296388

Scaling the features improved the performance of the support vector machine. After feature scaling, the model is no longer over-fitting to the training set (compare the ROC-AUC of 0.881 for the model on unscaled features vs. 0.68). In addition, the ROC-AUC for the testing set increased as well (0.66 vs 0.68).

1.3.3 K-Nearest Neighbours

```
[14]: # Model trained with unscaled features.

# The model.
KNN = KNeighborsClassifier(n_neighbors=5)

# Train the model.
KNN.fit(X_train, y_train)

# Evaluate performance.
print('Train set')
pred = KNN.predict_proba(X_train)
print('KNN roc-auc: {}'.format(roc_auc_score(y_train, pred[:,1])))
print('Test set')
pred = KNN.predict_proba(X_test)
print('KNN roc-auc: {}'.format(roc_auc_score(y_test, pred[:,1])))
```

Train set
KNN roc-auc: 0.8149809549207755

Test set
KNN roc-auc: 0.6865632431834522

```
[15]: # Model trained with scaled data.

# The model.
KNN = KNeighborsClassifier(n_neighbors=5)

# Train the model.
KNN.fit(X_train_scaled, y_train)

# Evaluate performance.
print('Train set')
pred = KNN.predict_proba(X_train_scaled)
print('KNN roc-auc: {}'.format(roc_auc_score(y_train, pred[:,1])))
print('Test set')
pred = KNN.predict_proba(X_test_scaled)
print('KNN roc-auc: {}'.format(roc_auc_score(y_test, pred[:,1])))
```

Train set
KNN roc-auc: 0.8260281072159968
Test set
KNN roc-auc: 0.7206183286322659

Feature scaling improved the performance of the KNN model. The model trained using scaled features shows better generalisation, that is, higher ROC-AUC for the testing set (0.72 vs. 0.69).

Both KNN methods overfit to the train set. Thus, we would need to change the parameters of the model or use fewer features to try and decrease over-fitting, which exceeds the purpose of this demonstration.

1.3.4 Random Forests

```
[16]: # Model trained with unscaled features.

# The model.
rf = RandomForestClassifier(n_estimators=200, random_state=39)

# Train the model.
rf.fit(X_train, y_train)

# Evaluate performance.
print('Train set')
pred = rf.predict_proba(X_train)
print('Random Forests roc-auc: {}'.format(roc_auc_score(y_train, pred[:, 1])))
print('Test set')
pred = rf.predict_proba(X_test)
print('Random Forests roc-auc: {}'.format(roc_auc_score(y_test, pred[:, 1])))
```

Train set

Random Forests roc-auc: 0.9866810238554083
Test set
Random Forests roc-auc: 0.7326751838946961

```
[17]: # Model trained with scaled features

# The model.
rf = RandomForestClassifier(n_estimators=200, random_state=39)

# Train the model.
rf.fit(X_train_scaled, y_train)

# Evaluate performance.
print('Train set')
pred = rf.predict_proba(X_train_scaled)
print('Random Forests roc-auc: {}'.format(roc_auc_score(y_train, pred[:,1])))
print('Test set')
pred = rf.predict_proba(X_test_scaled)
print('Random Forests roc-auc: {}'.format(roc_auc_score(y_test, pred[:,1])))
```

Train set
Random Forests roc-auc: 0.9867917218059866
Test set
Random Forests roc-auc: 0.7312510370001659

As expected, random forests show no change in performance regardless of whether they are trained on a dataset with scaled or unscaled features.

This model, in particular, is over-fitting to the training set. So we need to do some work to remove the over-fitting. That exceeds the scope of this demonstration.

1.3.5 AdaBoost

```
[18]: # Train Adaboost on non-scaled features.

# Adaboost
ada = AdaBoostClassifier(n_estimators=200, random_state=44)

# Train the model.
ada.fit(X_train, y_train)

# Evaluate model performance.
print('Train set')
pred = ada.predict_proba(X_train)
print('AdaBoost roc-auc: {}'.format(roc_auc_score(y_train, pred[:,1])))
print('Test set')
pred = ada.predict_proba(X_test)
print('AdaBoost roc-auc: {}'.format(roc_auc_score(y_test, pred[:,1])))
```

Train set

```
AdaBoost roc-auc: 0.7970629821021541
Test set
AdaBoost roc-auc: 0.7473867595818815
```

```
[19]: # Train Adaboost on scaled features.
```

```
# Adaboost.
ada = AdaBoostClassifier(n_estimators=200, random_state=44)

# Train the model.
ada.fit(X_train_scaled, y_train)

# Evaluate model performance.
print('Train set')
pred = ada.predict_proba(X_train_scaled)
print('AdaBoost roc-auc: {}'.format(roc_auc_score(y_train, pred[:,1])))
print('Test set')
pred = ada.predict_proba(X_test_scaled)
print('AdaBoost roc-auc: {}'.format(roc_auc_score(y_test, pred[:,1])))
```

```
Train set
AdaBoost roc-auc: 0.7970629821021541
Test set
AdaBoost roc-auc: 0.7475250262706707
```

As expected, AdaBoost shows no change in performance regardless of whether it is trained on a dataset with scaled or unscaled features.

That is all for this demonstration. I hope you enjoyed the notebook, and I'll see you in the next one.

```
[ ]:
```